CHMLTECH ONLINE JOURNAL · VOLUME 103, ISSUE 1 · RESEARCH ARTICLE

FrugalGPT and MultiModal-GPT: Advancements in Language Modeling

Quak Foo Lee
Chief Technology Officer, Chmltech Ltd.
Division of Artificial Intelligence and Machine Learning · Vancouver, BC, Canada

Published June 17, 2023 Revised November 9, 2026 19 min read

ABSTRACT

Large language models (LLMs) have transformed natural language processing, but the most capable models are expensive to query at scale and, in their text-only form, cannot reason about visual input. This paper reviews two contributions from May 2023 that address these limitations from opposite directions. FrugalGPT, proposed by Chen, Zaharia and Zou, is a framework for reducing the cost of using commercial LLM APIs; it describes three strategy families, prompt adaptation, LLM approximation and LLM cascade, of which the learned cascade was implemented and evaluated. The cascade queries inexpensive models first and escalates to costlier ones only when a small learned scorer rejects an answer, with the model ordering and acceptance thresholds optimized under a budget constraint. On three benchmark workloads it matched the best individual LLM (GPT-4) with up to 98% cost reduction, or improved accuracy over GPT-4 by about 4% at the same cost. MultiModal-GPT, proposed by Gong et al., is a vision-and-language dialogue model obtained by freezing OpenFlamingo and training low-rank adaptation (LoRA) modules on a unified mixture of vision-language and language-only instruction data. Its findings show that training-data quality governs dialogue behavior and that joint

language-only training improves conversation, although its evidence is qualitative. The paper formalizes both methods, corrects common misconceptions about them, compares their strengths and limitations, and outlines research directions. It concludes that small learned components around frozen foundation models can deliver substantial economic and functional gains when validated on the target workload.

Keywords: large language models; FrugalGPT; LLM cascade; MultiModal-GPT; vision-language models; LoRA; inference cost; instruction tuning

1. Introduction

Large language models (LLMs) built on the Transformer architecture [1] have become the dominant technology for natural language processing (NLP). Models such as GPT-3 demonstrated that a single autoregressive network with 175 billion parameters can perform translation, question answering and classification from a handful of in-context examples, without task-specific gradient updates [2]. Empirical scaling studies showed that test loss decreases approximately as a power law in model size, dataset size and training compute [3], which encouraged the development of ever larger systems, culminating in GPT-4, a model that accepts image and text inputs and reports human-level performance on a range of professional and academic examinations [4].

Two practical consequences of this trajectory motivate the present paper. First, the most capable models are typically accessed through commercial application programming interfaces (APIs) priced per token, and the cost of processing a large query workload with a top-tier model can be substantial; Chen, Zaharia and Zou reported that listed prices of the LLM APIs they surveyed in March 2023 differed by up to two orders of magnitude [5]. Second, text alone is an incomplete interface to the world: many practical tasks, from inspecting a photograph to reading a scanned document, require a model that can reason jointly over visual and linguistic input.

This article examines two contributions published in May 2023 that address these issues from different directions. FrugalGPT [5] is not a new language model but a framework of strategies for using existing LLM APIs more economically, of which the learned LLM cascade is implemented and evaluated. MultiModal-GPT [6] is a specific vision-and-language dialogue model obtained by parameter-efficient fine-tuning of the open-source OpenFlamingo model.

Both works are discussed from the viewpoint of mid-2023, with their technical mechanisms, reported evidence, strengths and limitations, and the research directions they open. The broader context of foundation models, in which a small number of large pretrained models are adapted to many downstream uses, frames both contributions [7].

The remainder of the paper is organized as follows. Section 2 reviews the Transformer language model and the cost structure of LLM APIs. Section 3 describes FrugalGPT. Section 4 describes MultiModal-GPT. Section 5 compares the two approaches, Section 6 discusses future directions, and Section 7 concludes.

2. Background

2.1 Transformer Language Models

A Transformer maps a sequence of token embeddings through stacked layers of multi-head self-attention and position-wise feed-forward networks [1]. The core operation is scaled dot-product attention, given in Eq. (1), in which queries Q, keys K and values V are linear projections of the layer input and d_k is the key dimension. The scaling by $\sqrt{d_k}$ prevents the dot products from growing large in magnitude and saturating the softmax.

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}(Q K^{\mathrm{T}} / \sqrt{d_k})\, V \tag{1}$$

Decoder-only language models such as GPT-3 apply a causal mask so that each position attends only to earlier positions, and they are pretrained by minimizing the negative log-likelihood of the next token [2]. For a token sequence $x_1, \ldots, x_T$ and model parameters θ, the training objective is the cross-entropy loss of Eq. (2).

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}) \tag{2}$$

Pretraining alone produces a model that continues text rather than one that follows instructions. Instruction tuning and reinforcement learning from human feedback align model outputs with user intent; Ouyang et al. reported that outputs of a 1.3-billion-parameter InstructGPT model were preferred by human labelers over those of the 175-billion-parameter GPT-3 [8]. Both FrugalGPT and MultiModal-GPT operate on such instruction-following models:

the former routes queries among them, and the latter applies the same objective of Eq. (2), restricted to response tokens, to a vision-language model.

2.2 Cost Structure of LLM APIs

FrugalGPT models the price of sending a query q to API i as the sum of three components: a cost proportional to the number of output tokens, a cost proportional to the number of prompt tokens and a fixed per-request fee [5]. Writing $\|\cdot\|$ for token count and $f_i(q)$ for the generated answer, the cost is given by Eq. (3), where $\tilde{c}_{i,2}$, $\tilde{c}_{i,1}$ and $\tilde{c}_{i,0}$ are provider-specific constants.

$$c_i(q) = \tilde{c}_{i,2}\,\|f_i(q)\| + \tilde{c}_{i,1}\,\|q\| + \tilde{c}_{i,0} \tag{3}$$

Eq. (3) makes the available levers explicit. The prompt-length term can be reduced by shortening prompts; the output-length and per-request terms can be avoided entirely by not calling an expensive API; and the constants themselves can be lowered by choosing a cheaper provider for a given query. The twelve APIs studied by Chen et al. ranged from small open models served commercially, such as GPT-J (6 billion parameters) through TextSynth, to GPT-3 (175 billion parameters) and GPT-4 from OpenAI, with intermediate offerings from AI21, Cohere and ForeFront AI [5]. Because few-shot prompting places several worked examples in every request [2], the prompt term often dominates the bill for classification-style workloads.

3. FrugalGPT

3.1 Problem Formulation

FrugalGPT addresses a budget-constrained setting: a user has a stream of queries, a set of LLM APIs with different prices and capabilities, and an average budget b per query, and wishes to maximize answer quality subject to that budget [5]. The authors grouped cost-reduction methods into three families, summarized in Table 1. It is important to distinguish the conceptual framework from the evaluated system: prompt adaptation and LLM approximation are described in the paper as directions, whereas the LLM cascade is the strategy that the authors implemented, named FrugalGPT, and evaluated empirically.

Table 1. The three families of cost-reduction strategies described by Chen et al. [5].

Strategy	Mechanism	Cost term reduced (Eq. 3)	Status in the original paper
Prompt selection	Keep only a subset of in-context examples relevant to each query	Prompt length $\|q\|$	Described, not evaluated
Query concatenation	Pack several queries into one request so the shared prompt is sent once	Prompt length and per-request fee	Described, not evaluated
Completion cache	Store answers and reuse them for identical or similar future queries	All terms (API call avoided)	Described, not evaluated
Model fine-tuning	Fine-tune a cheaper model on responses collected from an expensive LLM	Constants $\tilde{c}$ (cheaper model used)	Described, not evaluated
LLM cascade	Query APIs in sequence and stop when a learned scorer accepts an answer	Expected number of expensive calls	Implemented and evaluated

Source: [5].

3.2 Prompt Adaptation

Prompt adaptation in FrugalGPT targets the prompt-length term of Eq. (3) [5]. In *prompt selection*, a long few-shot prompt is replaced by a short one containing only the examples most useful for the current query, so that each request carries fewer tokens. The open problem is choosing examples without degrading accuracy, since the benefit of in-context examples in few-shot learning is well documented [2]. In *query concatenation*, several queries are combined into one request, with examples demonstrating how to answer multiple questions at once; the common prompt is then paid for once rather than once per query [5].

This is a narrower and more specific idea than generic prompt engineering. For example, a sentiment-classification service built on a few-shot prompt with eight labeled reviews pays for those eight reviews on every request. Prompt selection would retain, say, the two reviews most similar to the incoming one, and query concatenation would classify ten incoming

reviews within a single request. Neither technique changes the model; both reduce the tokens billed. Their effect on accuracy is task-dependent and was not quantified in the FrugalGPT paper [5].

3.3 LLM Approximation

LLM approximation replaces calls to an expensive model with a cheaper surrogate that behaves similarly on the target workload [5]. The first variant is a *completion cache*: answers returned by the API are stored locally, and a new query that matches or is sufficiently similar to a stored query is answered from the cache, which is most effective for workloads with repeated queries such as search. The second variant is *model fine-tuning*: responses of an expensive LLM are collected on a set of queries, a smaller and cheaper model is fine-tuned on these query–response pairs, and the smaller model then serves new queries. The authors noted that this can also lower latency because the fine-tuned model may need shorter prompts [5].

Model fine-tuning in this sense is a form of knowledge distillation, in which a compact student model is trained to reproduce the outputs of a larger teacher [9]. Distillation has a strong empirical record; DistilBERT, for example, is 40% smaller and 60% faster than BERT while retaining 97% of its language-understanding performance [10]. The trade-off is that the student inherits the teacher's errors, generalizes only within the distribution of the collected queries, and requires the user to host or pay for the fine-tuned model. Licensing terms of commercial APIs may also restrict using their outputs to train competing models, a practical constraint that deployments must review.

3.4 LLM Cascade

The cascade is the core of FrugalGPT [5]. Given an ordered list L of m APIs and a threshold vector τ, a query is first sent to L_1. A generation scoring function $g(q, a)$, taking values between 0 and 1, estimates the reliability of the returned answer a; if the score meets the threshold τ_1, the answer is returned, otherwise the query is forwarded to L_2, and so on. In the authors' implementation the scorer is a DistilBERT model [10] trained as a regressor to predict whether a query–answer pair is correct, chosen because it is far cheaper to run than the LLMs being routed [5]. The stopping stage z is defined by Eq. (4); if no earlier stage is accepted, the answer of the last API in the list is returned.

$$z = \min \{\, i : g(q, f_{L_i}(q)) \geq \tau_i \,\} \tag{4}$$

Learning the cascade means choosing L and τ to maximize the expected reward r, which measures agreement between the returned answer and the reference answer a, subject to the expected cumulative cost of all APIs queried up to stage z not exceeding the budget, as in Eq. (5) [5].

$$\max_{L,\,\tau} \mathbb{E}[r(a, f_{L_z}(q))] \text{ subject to } \mathbb{E}[\Sigma_{i=1}^{z} c_{L_i}(q)] \leq b \tag{5}$$

Eq. (5) is a mixed-integer problem: the ordering of APIs is discrete and the thresholds are continuous. Chen et al. solved it approximately with a specialized optimizer that prunes candidate API lists whose members rarely disagree and approximates the objective by interpolation from a small number of samples; the experiments used cascades of length three [5]. Two features of this design correct a common misreading. First, the later, more expensive model does not refine or fine-tune the output of the earlier one; it answers the query afresh, and only one answer is returned. Second, no LLM is trained: the learned components are the small scorer, the API ordering and the thresholds, fitted on a labeled sample of the workload. Figure 1 traces a query through the three-stage cascade that Chen et al. learned for HEADLINES: every rejection adds the cost of a further, more expensive call, and the last stage is returned without scoring.

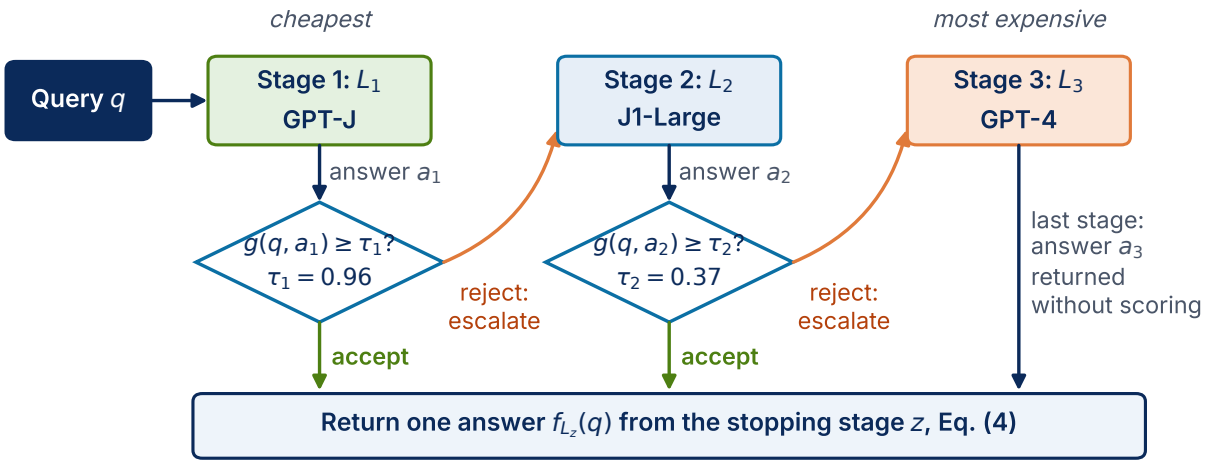


Figure 1. The FrugalGPT LLM cascade of Eqs. (4) and (5), shown with the policy learned for the HEADLINES dataset (GPT-J, then J1-Large, then GPT-4, with thresholds $\tau_1 = 0.96$ and $\tau_2 = 0.37$). Each model answers the query independently; the scorer g only decides whether to accept that answer or escalate.

Drawn by the author from Eqs. (4) and (5) and the HEADLINES case reported in [5].

The cascade works because LLMs make different errors. The authors measured how often a cheaper API answers correctly when a stronger one fails; on the HEADLINES dataset, small models such as GPT-J, J1-Large or GPT-Curie were correct on roughly 6% of queries that GPT-4 answered incorrectly, and on COQA, GPT-3 was correct on about 13% of queries where GPT-4 erred [5]. A cascade that accepts confident cheap answers therefore saves cost on easy queries and can, on some datasets, exceed the accuracy of the strongest single model.

3.5 Reported Results

FrugalGPT was evaluated on three datasets: HEADLINES (predicting gold-price movement from financial news headlines, 10,000 examples), OVERRULING (classifying whether a legal sentence overrules a precedent, 2,400 examples) and COQA (conversational reading comprehension, 7,982 examples) [5]. The abstract reports that FrugalGPT can match the performance of the best individual LLM, GPT-4, with up to 98% cost reduction, or improve accuracy over GPT-4 by about 4% at the same cost. Table 2 lists the per-dataset cost reductions at matched performance.

Table 2. Cost to match the best individual LLM on each dataset, as reported by Chen et al. [5].

Dataset	Domain	Best individual LLM	Cost of best LLM (USD)	FrugalGPT cost (USD)	Cost reduction
HEADLINES	Finance	GPT-4	33.1	0.6	98.3%
OVERRULING	Law	GPT-4	9.7	2.6	73.3%
COQA	Reading comprehension	GPT-3	72.5	29.6	59.2%

Costs are for the evaluation workload at March 2023 list prices. Source: [5].

A worked case on HEADLINES illustrates the learned policy. With a budget of one fifth of the cost of running GPT-4 on every query, the learned cascade queried GPT-J first, then J1-Large, then GPT-4, with acceptance thresholds of 0.96 and 0.37 for the first two stages. Accuracy rose from 0.857 for GPT-4 alone to 0.872, at roughly 80% lower cost [5]. On OVERRULING the cascade improved accuracy by about one percentage point while reducing cost by 73% relative to GPT-4. These are results on three benchmark workloads with prices current in early 2023; they demonstrate the mechanism rather than establishing a universal saving. Figure 2 sets these results side by side: the saving at matched performance ranges from 98.3% on HEADLINES to 59.2% on COQA, and at one fifth of GPT-4's cost the HEADLINES cascade is more accurate than GPT-4, not merely cheaper.

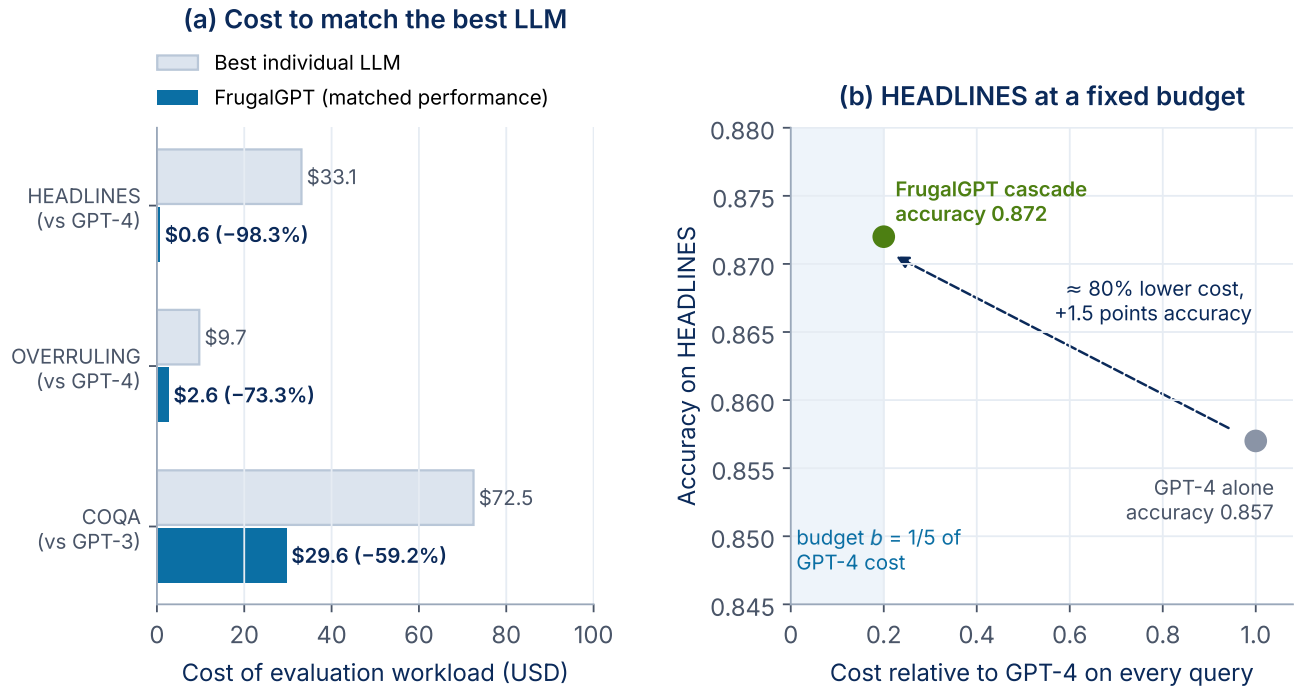


Figure 2. Reported cost and accuracy of FrugalGPT. (a) Cost of the evaluation workload for the best individual LLM and for FrugalGPT at matched performance (Table 2). (b) HEADLINES: GPT-4 on every query against the learned cascade at a budget of one fifth of GPT-4's cost.

Drawn by the author from Table 2 and the HEADLINES case in [5].

FrugalGPT's savings come from not calling the expensive model on queries that a cheap model, checked by a small learned scorer, already answers reliably. The achievable saving depends on the workload's mix of easy and hard queries and on the price gap between APIs.

3.6 Strengths and Limitations

The strengths of the approach follow directly from its design:

- **Cost reduction at controlled quality.** The budget is an explicit constraint in Eq. (5), so a user can trade cost against accuracy along a measured frontier rather than by guesswork [5].
- **Provider independence.** The method treats LLMs as black-box APIs and needs only their outputs, so new models can be added to the candidate pool without access to their weights.

- **Low adaptation cost.** Only a small scorer and a routing policy are trained; no LLM is trained or fine-tuned in the cascade itself.

The limitations stated by the authors, and some that follow from the method, are equally concrete [5]:

- **Labeled data and distribution shift.** Learning the scorer and thresholds requires labeled examples drawn from the same or a similar distribution as the production queries; performance may degrade if the query mix changes.
- **Upfront cost.** Fitting the cascade requires querying every candidate API on the training sample, which pays off only for sufficiently large workloads.
- **Latency.** A query rejected at early stages incurs the latency of several sequential calls; the method does not optimize latency, fairness or privacy.
- **Scorer reliability.** The scorer must detect wrong answers; for open-ended generation, where correctness is hard to judge, this is considerably harder than for classification, and uncertainty in risk-critical applications needs further study.

4. MultiModal-GPT

4.1 Vision-Language Foundations

Modern vision-language models combine a pretrained image encoder with a pretrained language model. CLIP provided a widely used encoder, trained contrastively on 400 million image–text pairs so that image and caption embeddings align in a shared space [11]. Flamingo, from DeepMind, connected a frozen vision encoder to a frozen large language model through two trainable components: a Perceiver Resampler that converts a variable number of visual features into a small fixed set of visual tokens, and gated cross-attention layers interleaved between the frozen language-model layers [12]. Trained on interleaved image–text web data, Flamingo set new few-shot results across many image and video understanding benchmarks. OpenFlamingo is an open-source reproduction of this architecture [13], and the language decoder used by MultiModal-GPT is LLaMA [6], an openly released family of foundation models from 7 to 65 billion parameters [14].

4.2 Architecture

MultiModal-GPT, introduced by Gong et al. [6], is a dialogue model fine-tuned from OpenFlamingo. It consists of a CLIP-based vision encoder, a Perceiver Resampler that receives spatial features from the encoder, and a LLaMA language decoder that attends to the resampled visual tokens through gated cross-attention [6]. In the Flamingo design, each gated cross-attention block adds its output to the residual stream through a learnable scalar gate, as in Eq. (6), where x is the language hidden state, V the visual tokens and α a learnable parameter initialized to zero, so that at initialization the network reproduces the frozen language model exactly [12].

$$y = x + \tanh(\alpha) \cdot \mathrm{XAttn}(x, V) \tag{6}$$

The model processes images and text; it does not accept audio, and the paper does not demonstrate video input [6]. For reference, the later OpenFlamingo technical report documents models of 3 to 9 billion parameters that pair a CLIP ViT-L/14 encoder with MPT or RedPajama decoders, training only the cross-attention modules and the Perceiver Resampler while keeping both pretrained backbones frozen [13].

4.3 Parameter-Efficient Fine-Tuning with LoRA

Rather than updating the full OpenFlamingo model, Gong et al. froze it entirely and added low-rank adaptation (LoRA) modules to the self-attention, cross-attention and feed-forward parts of the language decoder [6]. LoRA represents the update to a pretrained weight matrix $W_0 \in \mathbb{R}^{d \times k}$ as the product of two small matrices, as in Eq. (7), where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, the rank $r \ll \min(d, k)$, and α_L is a scaling constant [15]. Figure 3 shows where the adapters sit: the vision encoder, Perceiver Resampler and every decoder sublayer stay frozen, and only the small LoRA matrices beside the cross-attention, self-attention and feed-forward sublayers are trained.

$$h = W_0 x + (\alpha_L / r)\, BAx \tag{7}$$

Only A and B are trained; B is initialized to zero so that training starts from the pretrained model. Hu et al. reported that, for GPT-3 175B, LoRA reduced the number of trainable parameters by a factor of about 10,000 and GPU memory requirements by a factor of three relative to full fine-tuning, with no additional inference latency once the low-rank product is

merged into W_0 [15]. Subsequent work combined LoRA with 4-bit quantization of the frozen weights, making it possible to fine-tune a 65-billion-parameter model on a single 48 GB GPU [16]. Parameter efficiency is what allowed MultiModal-GPT to be trained in one epoch on eight A100 GPUs [6].

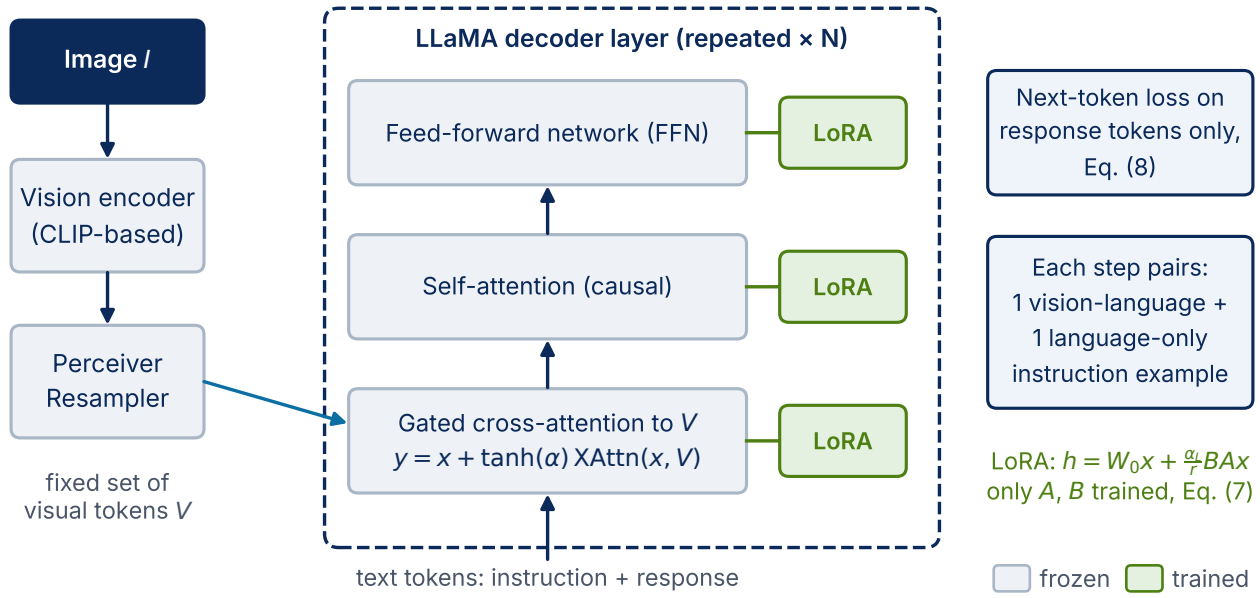


Figure 3. MultiModal-GPT architecture and training recipe. Visual tokens from the frozen CLIP-based encoder and Perceiver Resampler enter each frozen LLaMA decoder layer through gated cross-attention, Eq. (6); LoRA adapters, Eq. (7), are the only trained parameters, and the loss of Eq. (8) is computed on response tokens only.

Adapted from [6] and [12].

4.4 Instruction Data and Training Objective

Gong et al. built a unified instruction template for both language-only and vision-language data, so that the two could be trained jointly in the same format [6]. The vision-language data comprised the instruction-following sets released with LLaVA [17] and MiniGPT-4 [18], a 5,000-example sample of A-OKVQA [19], and 512 examples each from COCO Caption [20] and OCR-VQA. Because COCO captions lack instructions, GPT-4 was used to generate varied instruction phrasings for them. The language-only data comprised the Dolly 15k corpus and the Alpaca-GPT4 instruction data of Peng et al. [6, 21]. Each training iteration paired one vision-language and one language-only example, with gradient accumulation giving an effective batch size of 256 and a cosine-scheduled learning rate of 1×10^{-5} [6].

The training objective is the next-token cross-entropy of Eq. (2), computed only over the tokens of the response and the end-of-sequence token, so the model is not penalized for failing to predict the user's instruction or the image placeholder [6]. With R denoting the set of response positions, the loss is given by Eq. (8), where I is the image.

$$\mathcal{L}_{\text{resp}}(\theta) = -\Sigma_{t \in R} \log p_\theta(x_t \mid x_{<t}, I) \tag{8}$$

Two findings on data are the paper's most transferable lessons. First, data quality dominated: the authors removed VQA v2.0, OK-VQA, GQA, CLEVR and NLVR from the training mixture because their one- or two-word answers caused the model to answer every request tersely [6]. Second, joint training with language-only instruction data improved dialogue performance, which the authors attributed to preserving the conversational ability of the language model while it learned to use visual input [6].

4.5 Demonstrated Capabilities and Evidence

MultiModal-GPT supports multi-round dialogue about an image. The published demonstrations include identifying a dish and then supplying a recipe and restaurant suggestions, counting people and describing their activity, recognizing a film poster and its studio, reading text on a book cover and naming the author, and describing a landscape and inferring the season [6]. For example, given a photograph of a football match and the instruction to describe it, the model can produce a caption such as "a group of people playing soccer on a field" and then answer follow-up questions about the number of players or the setting.

The evidence is qualitative. The paper presents curated demonstrations rather than scores on standardized benchmarks, so it does not establish that MultiModal-GPT outperforms contemporaneous models such as LLaVA [17] or MiniGPT-4 [18] on any quantitative measure. Its contribution is better understood as an open, reproducible recipe — frozen OpenFlamingo, LoRA adapters and a curated, mixed instruction dataset — and a set of practical observations about instruction data quality.

4.6 Strengths and Limitations

- **Grounded dialogue.** Conditioning on visual tokens lets the model answer questions whose answers are not in the text, such as counting objects or reading text in an image [6].

- **Efficiency of adaptation.** Because the backbone is frozen and only LoRA weights are trained, adaptation is inexpensive compared with full fine-tuning [15].
- **Openness.** The model, code and data recipe are public, which supports reproduction and extension.
- **Limited modality scope.** The model handles images and text only; claims that it processes audio or video are not supported by the paper.
- **Hallucination and inherited bias.** As with other vision-language models built on LLMs, answers can be fluent but unfaithful to the image, and biases of the pretrained backbones and of GPT-4-generated instruction data are inherited.
- **Evaluation gap.** Without quantitative benchmarking, the reliability of the model for a given application must be established by the user.

5. Comparative Discussion

The two contributions address different constraints of the LLM ecosystem but share a common philosophy: both reuse large pretrained models rather than training new ones, and both obtain their gains through small learned components placed around frozen foundations [7]. FrugalGPT learns a scorer and a routing policy around black-box APIs; MultiModal-GPT learns low-rank adapters inside a frozen open model. The resulting systems are inexpensive to build relative to the models they depend on.

They differ in what they optimize. FrugalGPT treats capability as fixed and optimizes cost, with accuracy gains arising as a side effect of model diversity [5]. MultiModal-GPT treats cost as secondary and extends capability to a new input modality [6]. They also differ in the evidence offered: FrugalGPT reports quantitative cost–accuracy curves on three labeled datasets, whereas MultiModal-GPT reports qualitative demonstrations.

The ideas are complementary. A multimodal service could apply a cascade in which a small open vision-language model answers routine image queries and a stronger, more expensive multimodal model such as GPT-4 [4] is called only when a scorer rejects the cheaper answer. The obstacle is the scorer: judging the correctness of free-form answers about images is harder than judging a class label, and a cascade is only as good as its acceptance test.

6. Future Directions

Several research directions follow from the limitations identified above:

- **Better optimization of cascades.** More efficient solvers for Eq. (5), query-dependent ordering of APIs instead of a fixed list, and scorers calibrated for open-ended generation would broaden the applicability of FrugalGPT-style routing [5].
- **Empirical study of prompt adaptation and approximation.** Prompt selection, query concatenation, caching and distillation were proposed but not evaluated in the original work; quantifying their cost–accuracy trade-offs, alone and combined with cascades, is an open task [5, 9].
- **Adaptation to changing pools and prices.** Because API prices and model capabilities change frequently, cascades should be re-fitted, or adapted online, as candidates are added or retired.
- **Richer multimodal fusion and evaluation.** Extending MultiModal-GPT-style models to video and audio, and evaluating them on standardized benchmarks rather than demonstrations, would make capability claims testable [12, 6].
- **Responsible deployment.** Both approaches inherit the risks of the underlying models, including discrimination, misinformation and privacy harms; Weidinger et al. organized such risks into six areas that can serve as a checklist for deployment [22].

Note on subsequent developments (2026 revision). The body of this paper reflects the state of the field in mid-2023. Since then, learned routing between models has become an established research topic; RouteLLM, for example, trained routers on human preference data and reported cost reductions of more than a factor of two in some settings without loss of response quality [23]. In vision-language modeling, LLaVA-1.5 showed that a simple projection from a CLIP encoder into an LLM, trained on about 1.2 million publicly available examples, achieved state-of-the-art results across eleven benchmarks at the time of its publication [24]. These developments are consistent with, rather than a replacement for, the principles discussed above: route by difficulty, and adapt frozen foundations with small, well-curated training sets.

7. Conclusion

FrugalGPT and MultiModal-GPT illustrate two responses to the maturing of large language models in 2023. FrugalGPT is a framework for using existing LLM APIs economically. Of its three strategy families, the learned LLM cascade was implemented and evaluated: by querying cheaper models first and escalating only when a small learned scorer rejects their answers, it matched the performance of the best individual LLM at up to 98% lower cost on

one benchmark, and improved accuracy over GPT-4 by about 4% at equal cost, on three classification and reading-comprehension workloads [5]. Prompt adaptation and LLM approximation remain promising but unquantified in that work.

MultiModal-GPT is a specific open vision-language dialogue model, obtained by freezing OpenFlamingo and training LoRA adapters on a curated mixture of vision-language and language-only instruction data [6, 15]. Its main lessons are methodological: data quality governs dialogue behavior, and joint language-only training preserves conversational ability. Its evidence is qualitative and its scope is limited to images and text.

Both approaches show that substantial value can be extracted from existing foundation models through small, well-designed components. Their practical adoption depends on the same disciplines: measuring performance on the user's own workload, monitoring for distribution shift, and assessing the ethical and safety risks the underlying models carry [22].

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30. arXiv:1706.03762. arxiv.org/abs/1706.03762
2. Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877–1901. arXiv:2005.14165. arxiv.org/abs/2005.14165
3. Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. arXiv preprint arXiv:2001.08361. arxiv.org/abs/2001.08361
4. OpenAI. (2023). GPT-4 technical report. arXiv preprint arXiv:2303.08774. arxiv.org/abs/2303.08774
5. Chen, L., Zaharia, M., & Zou, J. (2023). FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv preprint arXiv:2305.05176. arxiv.org/abs/2305.05176
6. Gong, T., Lyu, C., Zhang, S., Wang, Y., Zheng, M., Zhao, Q., Liu, K., Zhang, W., Luo, P., & Chen, K. (2023). MultiModal-GPT: A vision and language model for dialogue with humans. arXiv preprint arXiv:2305.04790. arxiv.org/abs/2305.04790
7. Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., et al. (2021). On the opportunities and risks of foundation models. arXiv preprint arXiv:2108.07258. arxiv.org/abs/2108.07258

8. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., et al. (2022). Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 35. arXiv:2203.02155. arxiv.org/abs/2203.02155
9. Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531. arxiv.org/abs/1503.02531
10. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108. arxiv.org/abs/1910.01108
11. Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., et al. (2021). Learning transferable visual models from natural language supervision. Proceedings of the 38th International Conference on Machine Learning (ICML), PMLR 139, 8748–8763. arXiv:2103.00020. arxiv.org/abs/2103.00020
12. Alayrac, J.-B., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., et al. (2022). Flamingo: A visual language model for few-shot learning. Advances in Neural Information Processing Systems, 35. arXiv:2204.14198. arxiv.org/abs/2204.14198
13. Awadalla, A., Gao, I., Gardner, J., Hessel, J., Hanafy, Y., Zhu, W., et al. (2023). OpenFlamingo: An open-source framework for training large autoregressive vision-language models. arXiv preprint arXiv:2308.01390. arxiv.org/abs/2308.01390
14. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., et al. (2023). LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971. arxiv.org/abs/2302.13971
15. Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685 (published at ICLR 2022). arxiv.org/abs/2106.09685
16. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. arXiv preprint arXiv:2305.14314. arxiv.org/abs/2305.14314
17. Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). Visual instruction tuning. arXiv preprint arXiv:2304.08485. arxiv.org/abs/2304.08485
18. Zhu, D., Chen, J., Shen, X., Li, X., & Elhoseiny, M. (2023). MiniGPT-4: Enhancing vision-language understanding with advanced large language models. arXiv preprint arXiv:2304.10592. arxiv.org/abs/2304.10592
19. Schwenk, D., Khandelwal, A., Clark, C., Marino, K., & Mottaghi, R. (2022). A-OKVQA: A benchmark for visual question answering using world knowledge. European Conference on Computer Vision (ECCV). arXiv:2206.01718. arxiv.org/abs/2206.01718
20. Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C. L., & Dollár, P. (2014). Microsoft COCO: Common objects in context. European

Conference on Computer Vision (ECCV), 740–755. arXiv:1405.0312. arxiv.org/abs/1405.0312

21. Peng, B., Li, C., He, P., Galley, M., & Gao, J. (2023). Instruction tuning with GPT-4. arXiv preprint arXiv:2304.03277. arxiv.org/abs/2304.03277
22. Weidinger, L., Mellor, J., Rauh, M., Griffin, C., Uesato, J., Huang, P.-S., et al. (2021). Ethical and social risks of harm from language models. arXiv preprint arXiv:2112.04359. arxiv.org/abs/2112.04359
23. Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., & Stoica, I. (2024). RouteLLM: Learning to route LLMs with preference data. arXiv preprint arXiv:2406.18665. arxiv.org/abs/2406.18665
24. Liu, H., Li, C., Li, Y., & Lee, Y. J. (2023). Improved baselines with visual instruction tuning. arXiv preprint arXiv:2310.03744. arxiv.org/abs/2310.03744

How to cite this article

Lee, Q. F. (2023). FrugalGPT and MultiModal-GPT: Advancements in Language Modeling. *Chmltech Online Journal*, *103*(1). Revised November 9, 2026. https://www.chmltech.com/post/chmltech-online-journal-volume-103-issue-1

This article was first published on June 17, 2023. The revised edition of November 9, 2026 expands the original, corrects technical errors and adds verified references.

Ready to find your next improvement?

Book a strategy call. We will talk through your goals, pain points and where AI, systems or process work can make the biggest difference.

Book a Strategy Call

Or email **info@chmltech.com**